SKILL WEEK-12

SESSION-23

Task-1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze 4 Performance.

Source code:

```
 GNU nano 8.7.1                    pipeline_test.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    for (int i = 1; i <= 10; i++)
    {
        printf("Pipeline process: %d\n", i);
        fflush(stdout);
        usleep(200000);
    }

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$       pipeline_test.c
sathwika@DESKTOP-AA8K5MG:~$      pipeline_test.c -o pipeline_test
sathwika@DESKTOP-AA8K5MG:~$      peline_test | grep "process" | wc -l
10
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

Large pipelines and multiple background jobs were successfully executed. Process activity and system resources were monitored, pipeline failures were tested, and execution time was measured using Linux utilities.

Task-2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design 4 Decisions, Review Documentation.

Source code:

```
 GNU nano 8.7.1                          main.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[100];
    ssize_t bytes;

    printf("Program started\n");

    fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    bytes = read(fd, buffer, sizeof(buffer) - 1);

    if (bytes < 0) {
        perror("read");
        close(fd);
        return 1;
    }

    buffer[bytes] = '\0';

    printf("File contents: %s\n", buffer);

    close(fd);

    printf("Program completed\n");

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$       documentation_demo
cd documentation_demo
nano main.c
sathwika@DESKTOP-AA8K5MG:~$       tation_demo$ echo "Hello from Ubuntu Linu
x" > input.txt
sathwika@DESKTOP-AA8K5MG:~$       tation_demo$ gcc main.c -o documentation_
demo
sathwika@DESKTOP-AA8K5MG:~$       tation_demo$ ./documentation_demo
Program started
File contents: Hello from Ubuntu Linux

Program completed
sathwika@DESKTOP-AA8K5MG:~$       tation_demo$
```

Observation:

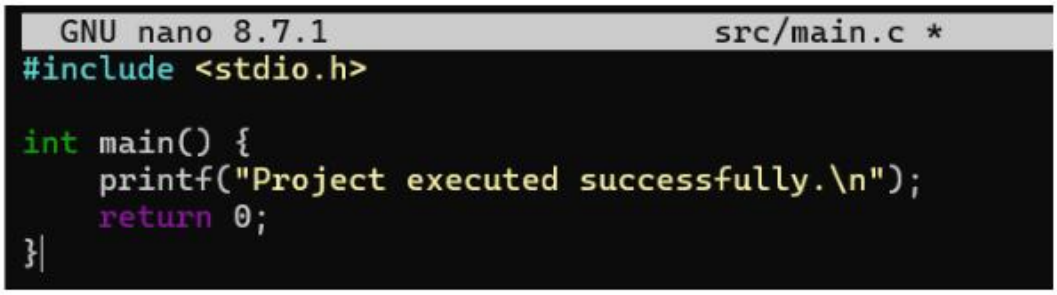
The program successfully demonstrates the architecture and execution flow of a Linux file-reading application. The execution begins from main(), opens the input file using open(), reads its contents using read(), displays the data, and finally releases the file descriptor using close(). Using strace, the system calls involved in program execution can be observed. The flow diagram and design decisions provide a clear representation of the program architecture and execution path.

SESSION-24

Task-1: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

Source code:

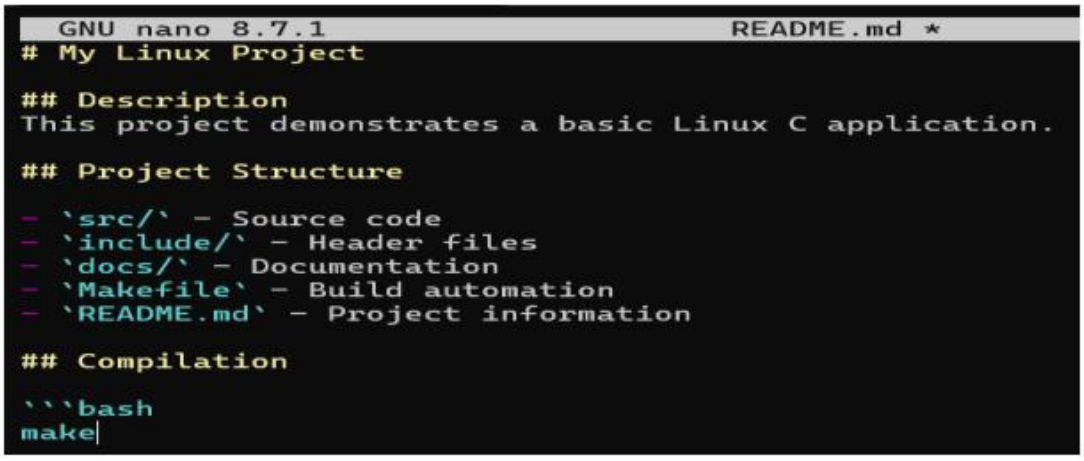
Nano src/main.c:



Nano Makefile:



nano README.md:



Output:

```
sathwika@DESKTOP-AA8K5MG:~$       nano src/main.c
sathwika@DESKTOP-AA8K5MG:~$       nano Makefile
sathwika@DESKTOP-AA8K5MG:~$       make
gcc -Wall -Wextra src/main.c -o my_program
sathwika@DESKTOP-AA8K5MG:~$       ./my_program
Project executed successfully.
sathwika@DESKTOP-AA8K5MG:~$       make run
./my_program
Project executed successfully.
sathwika@DESKTOP-AA8K5MG:~$       nano README.md
sathwika@DESKTOP-AA8K5MG:~$       ./my_program
Project executed successfully.
sathwika@DESKTOP-AA8K5MG:~$       make clean
rm -f my_program
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

The source code was organized into separate directories, and a Makefile was created to automate compilation and execution. A README file was added to document the project, and the repository structure was verified using tree and Git commands. The project was successfully compiled and executed, and release tag v1.0.0 was created to identify the validated version of the deliverables.

Task-2: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

Source code:

```
  GNU nano 8.7.1                    src/main.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    printf("\nSignal %d received.\n", sig);
}

int main() {
    signal(SIGINT, handle_signal);

    printf("=== Final Project Demo ===\n");
    printf("PID: %d\n", getpid());

    printf("Project is running...\n");

    for (int i = 1; i <= 10; i++) {
        printf("Step %d completed\n", i);
        sleep(2);
    }

    printf("Project execution completed.\n");

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$       ~/my_project
nano src/main.c
-bash: cd: /home/avulasanjana/my_project: No such file or directory
sathwika@DESKTOP-AA8K5MG:~$      ke clean
make
rm -f my_program
gcc -Wall -Wextra src/main.c -o my_program
sathwika@DESKTOP-AA8K5MG:~$      my_program
=== Final Project Demo ===
PID: 506
Project is running...
Step 1 completed
Step 2 completed
Step 3 completed
Step 4 completed
Step 5 completed
Step 6 completed
Step 7 completed
Step 8 completed
Step 9 completed
Step 10 completed
Project execution completed.
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

The final project was successfully demonstrated through multiple test scenarios. The project was compiled and executed using the Makefile, pipelines were demonstrated using Linux commands, background execution was verified, and signal handling was tested using SIGINT. Git status, commit history, project structure, and release tags were also verified. The final review confirmed that the major project features and deliverables were working as expected.